

CS3014 Estructuras de Datos Avanzadas

Laboratorio - Semana 9 - Sesión 1

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

20 de mayo de 2026

¿Qué aprenderemos hoy?



¿Qué aprenderemos hoy?



- ▶ Modelos computacional RAM AC^0

¿Qué aprenderemos hoy?



- ▶ Modelos computacional RAM AC^0
- ▶ Fusion Trees

Modelos computacional RAM AC^0

Modelos computacional RAM AC^0



Modelo RAM AC^0

Modelos computacional RAM AC^0



Modelo RAM AC^0

- ▶ Permite solo operaciones que se puedan realizar en un circuito de profundidad $O(1)$ pero sin límite de fan-in y fan-out (compuertas que entran y salen, respectivamente).

Modelos computacional RAM AC^0



Modelo RAM AC^0

- ▶ Permite solo operaciones que se puedan realizar en un circuito de profundidad $O(1)$ pero sin límite de fan-in y fan-out (compuertas que entran y salen, respectivamente).
- ▶ Por su definición, no permite multiplicación.

Modelos computacional RAM AC^0



Modelo RAM AC^0

- ▶ Permite solo operaciones que se puedan realizar en un circuito de profundidad $O(1)$ pero sin límite de fan-in y fan-out (compuertas que entran y salen, respectivamente).
- ▶ Por su definición, no permite multiplicación.

Otras operaciones permitidas

Modelos computacional RAM AC^0



Modelo RAM AC^0

- ▶ Permite solo operaciones que se puedan realizar en un circuito de profundidad $O(1)$ pero sin límite de fan-in y fan-out (puertas que entran y salen, respectivamente).
- ▶ Por su definición, no permite multiplicación.

Otras operaciones permitidas

Operaciones bitwise pueden ser realizadas en $O(1)$.

Fusion Trees

Fusion Trees



Definición

Fusion Trees



Definición

Es una estructura de datos que nos permite consultar el sucesor/predecesor en $O(\log_w n)$.

Fusion Trees



Definición

Es una estructura de datos que nos permite consultar el sucesor/predecesor en $O(\log_w n)$.

Considerando este resultado y el de vEB y y-fast trees, tendríamos que la consulta de sucesor predecesor toma

$$\min \{O(\log w), O(\log_w n)\} = O(\sqrt{\log n})$$

Fusion Trees



Definición

Es una estructura de datos que nos permite consultar el sucesor/predecesor en $O(\log_w n)$.

Considerando este resultado y el de vEB y y-fast trees, tendríamos que la consulta de sucesor predecesor toma

$$\min \{O(\log w), O(\log_w n)\} = O(\sqrt{\log n})$$

Versiones

Fusion Trees



Definición

Es una estructura de datos que nos permite consultar el sucesor/predecesor en $O(\log_w n)$.

Considerando este resultado y el de vEB y y-fast trees, tendríamos que la consulta de sucesor predecesor toma

$$\min \{O(\log w), O(\log_w n)\} = O(\sqrt{\log n})$$

Versiones

Veremos la versión estática. Las dinámicas disponibles cumplen:

Fusion Trees



Definición

Es una estructura de datos que nos permite consultar el sucesor/predecesor en $O(\log_w n)$.

Considerando este resultado y el de vEB y y-fast trees, tendríamos que la consulta de sucesor predecesor toma

$$\min \{O(\log w), O(\log_w n)\} = O(\sqrt{\log n})$$

Versiones

Veremos la versión estática. Las dinámicas disponibles cumplen:

- $O(\log_w n + \log \log n)$ usando exponential trees.

Fusion Trees



Definición

Es una estructura de datos que nos permite consultar el sucesor/predecesor en $O(\log_w n)$.

Considerando este resultado y el de vEB y y-fast trees, tendríamos que la consulta de sucesor predecesor toma

$$\min \{O(\log w), O(\log_w n)\} = O(\sqrt{\log n})$$

Versiones

Veremos la versión estática. Las dinámicas disponibles cumplen:

- ▶ $O(\log_w n + \log \log n)$ usando exponential trees.
- ▶ $O(\log_w n)$ esperado usando hashing.

Estructura



Planteamiento inicial

Estructura



Planteamiento inicial

Consideraremos un B-Tree de orden $w^{\frac{1}{5}}$, así que cada nodo tiene a lo mucho $w^{\frac{1}{5}}$ hijos.

Estructura



Planteamiento inicial

Consideraremos un B-Tree de orden $w^{\frac{1}{5}}$, así que cada nodo tiene a lo mucho $w^{\frac{1}{5}}$ hijos.

Cada nodo maneja $w^{1+\frac{1}{5}}$ bits, así que no podemos buscar el hijo correspondiente en $O(1)$.

Estructura



Planteamiento inicial

Consideraremos un B-Tree de orden $w^{\frac{1}{5}}$, así que cada nodo tiene a lo mucho $w^{\frac{1}{5}}$ hijos.

Cada nodo maneja $w^{1+\frac{1}{5}}$ bits, así que no podemos buscar el hijo correspondiente en $O(1)$.

Solución - Nodos de fusion tree

Estructura



Planteamiento inicial

Consideraremos un B-Tree de orden $w^{\frac{1}{5}}$, así que cada nodo tiene a lo mucho $w^{\frac{1}{5}}$ hijos.

Cada nodo maneja $w^{1+\frac{1}{5}}$ bits, así que no podemos buscar el hijo correspondiente en $O(1)$.

Solución - Nodos de fusion tree

- ▶ Almacenamos $k = O(w^{\frac{1}{5}})$ valores $x_0 < x_1 < \dots < x_{k-1}$.

Estructura



Planteamiento inicial

Consideraremos un B-Tree de orden $w^{\frac{1}{5}}$, así que cada nodo tiene a lo mucho $w^{\frac{1}{5}}$ hijos.

Cada nodo maneja $w^{1+\frac{1}{5}}$ bits, así que no podemos buscar el hijo correspondiente en $O(1)$.

Solución - Nodos de fusion tree

- ▶ Almacenamos $k = O(w^{\frac{1}{5}})$ valores $x_0 < x_1 < \dots < x_{k-1}$.
- ▶ Debemos poder responder al sucesor o predecesor de un valor en $O(1)$.

Estructura



Planteamiento inicial

Consideraremos un B-Tree de orden $w^{\frac{1}{5}}$, así que cada nodo tiene a lo mucho $w^{\frac{1}{5}}$ hijos.

Cada nodo maneja $w^{1+\frac{1}{5}}$ bits, así que no podemos buscar el hijo correspondiente en $O(1)$.

Solución - Nodos de fusion tree

- ▶ Almacenamos $k = O(w^{\frac{1}{5}})$ valores $x_0 < x_1 < \dots < x_{k-1}$.
- ▶ Debemos poder responder al sucesor o predecesor de un valor en $O(1)$.
- ▶ Requeriremos $k^{O(1)}$ de preprocesamiento.

Sketching



¿Cómo distinguimos $k = O(w^{\frac{1}{5}})$ valores?

Sketching



¿Cómo distinguimos $k = O(w^{\frac{1}{5}})$ valores?

Podemos considerar un trie de las llaves con un alfabeto binario.

Sketching



¿Cómo distinguimos $k = O(w^{\frac{1}{5}})$ valores?

Podemos considerar un trie de las llaves con un alfabeto binario. Es posible reducir la cantidad total de nodos calculando el auxiliary/virtual tree del trie correspondiente y mapeamos los niveles involucrados.

Sketching



¿Cómo distinguimos $k = O(w^{\frac{1}{5}})$ valores?

Podemos considerar un trie de las llaves con un alfabeto binario. Es posible reducir la cantidad total de nodos calculando el auxiliary/virtual tree del trie correspondiente y mapeamos los niveles involucrados.

Consecuencias

Sketching



¿Cómo distinguimos $k = O(w^{\frac{1}{5}})$ valores?

Podemos considerar un trie de las llaves con un alfabeto binario. Es posible reducir la cantidad total de nodos calculando el auxiliary/virtual tree del trie correspondiente y mapeamos los niveles involucrados.

Consecuencias

Hay $k - 1$ nodos internos en el trie, así que nos interesan a lo mucho $k - 1$ niveles del mismo.

Sketching



¿Cómo distinguimos $k = O(w^{\frac{1}{5}})$ valores?

Podemos considerar un trie de las llaves con un alfabeto binario. Es posible reducir la cantidad total de nodos calculando el auxiliary/virtual tree del trie correspondiente y mapeamos los niveles involucrados.

Consecuencias

Hay $k - 1$ nodos internos en el trie, así que nos interesan a lo mucho $k - 1$ niveles del mismo.

Si almacenamos la información de esos $k - 1$ niveles, entonces manejaremos $O(w^{\frac{1}{5}})$ bits por cada llave.

Sketching



¿Cómo distinguimos $k = O(w^{\frac{1}{5}})$ valores?

Podemos considerar un trie de las llaves con un alfabeto binario. Es posible reducir la cantidad total de nodos calculando el auxiliary/virtual tree del trie correspondiente y mapeamos los niveles involucrados.

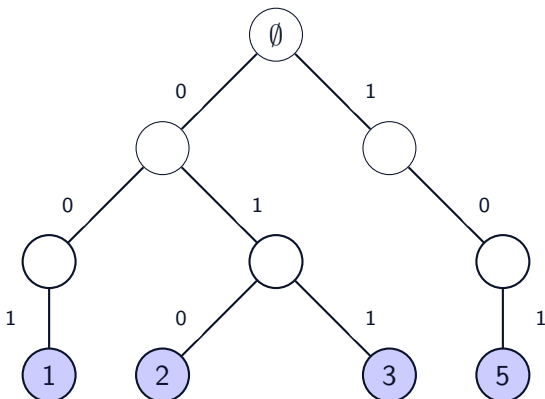
Consecuencias

Hay $k - 1$ nodos internos en el trie, así que nos interesan a lo mucho $k - 1$ niveles del mismo.

Si almacenamos la información de esos $k - 1$ niveles, entonces manejaremos $O(w^{\frac{1}{5}})$ bits por cada llave.

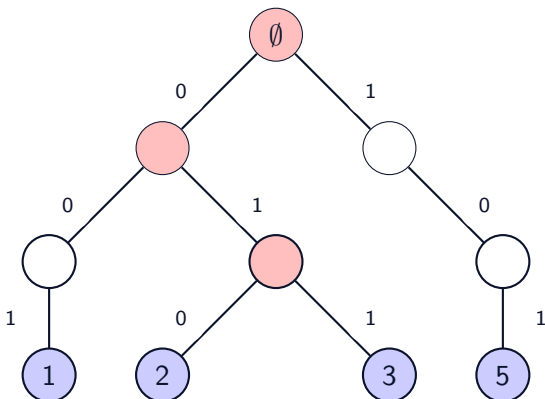
En total, manejaremos $O(w^{\frac{2}{5}})$ bits, lo cual si es posible de hacer en $O(1)$.

Trie binario



1 = 001, 2 = 010, 3 = 011, 5 = 101

Trie binario + Virtual Tree



1 = 001, 2 = 010, 3 = 011, 5 = 101

Sketching perfecto



Definición

Sketching perfecto



Definición

Consideraremos una secuencia de r enteros $b_0 > b_1 > \dots > b_{r-1}$ y cada llave la representaremos como una cadena de r bits en la cual el i -ésimo bit corresponderá al bit b_i de la llave.

Sketching perfecto



Definición

Consideraremos una secuencia de r enteros $b_0 > b_1 > \dots > b_{r-1}$ y cada llave la representaremos como una cadena de r bits en la cual el i -ésimo bit corresponderá al bit b_i de la llave.

Denotaremos a dicha cadena binaria como $\text{SKETCH}(x)$.

Sketching perfecto



Definición

Consideraremos una secuencia de r enteros $b_0 > b_1 > \dots > b_{r-1}$ y cada llave la representaremos como una cadena de r bits en la cual el i -ésimo bit corresponderá al bit b_i de la llave.

Denotaremos a dicha cadena binaria como $\text{SKETCH}(x)$.

Propiedad importante

Sketching perfecto



Definición

Consideraremos una secuencia de r enteros $b_0 > b_1 > \dots > b_{r-1}$ y cada llave la representaremos como una cadena de r bits en la cual el i -ésimo bit corresponderá al bit b_i de la llave.

Denotaremos a dicha cadena binaria como $\text{SKETCH}(x)$.

Propiedad importante

$$\text{SKETCH}(x_0) < \text{SKETCH}(x_1) < \dots < \text{SKETCH}(x_{k-1})$$

Sketching perfecto



Definición

Consideraremos una secuencia de r enteros $b_0 > b_1 > \dots > b_{r-1}$ y cada llave la representaremos como una cadena de r bits en la cual el i -ésimo bit corresponderá al bit b_i de la llave.

Denotaremos a dicha cadena binaria como $\text{SKETCH}(x)$.

Propiedad importante

$$\text{SKETCH}(x_0) < \text{SKETCH}(x_1) < \dots < \text{SKETCH}(x_{k-1})$$

Esto se cumple **solo para las llaves**.

Consultas



¿Cómo buscamos el sucesor o predecesor de un valor q ?

Consultas



¿Cómo buscamos el sucesor o predecesor de un valor q ?

Podemos pensar en calcular $\text{SKETCH}(q)$ y luego buscarlo en la estructura.

Consultas



¿Cómo buscamos el sucesor o predecesor de un valor q ?

Podemos pensar en calcular $\text{SKETCH}(q)$ y luego buscarlo en la estructura. Sorpresa: SKETCH preserva el orden entre las llaves, así que $\text{SKETCH}(q)$ podría caer en cualquier lugar, no necesariamente en su sucesor o predecesor.

Consultas



¿Cómo buscamos el sucesor o predecesor de un valor q ?

Podemos pensar en calcular $\text{SKETCH}(q)$ y luego buscarlo en la estructura. Sorpresa: SKETCH preserva el orden entre las llaves, así que $\text{SKETCH}(q)$ podría caer en cualquier lugar, no necesariamente en su sucesor o predecesor.

¿Qué hacemos?

Consultas



¿Cómo buscamos el sucesor o predecesor de un valor q ?

Podemos pensar en calcular $\text{SKETCH}(q)$ y luego buscarlo en la estructura. Sorpresa: SKETCH preserva el orden entre las llaves, así que $\text{SKETCH}(q)$ podría caer en cualquier lugar, no necesariamente en su sucesor o predecesor.

¿Qué hacemos?

Supongamos que $\text{SKETCH}(x_i) \leq \text{SKETCH}(q) < \text{SKETCH}(x_{i+1})$.

Consultas



¿Cómo buscamos el sucesor o predecesor de un valor q ?

Podemos pensar en calcular $\text{SKETCH}(q)$ y luego buscarlo en la estructura. Sorpresa: SKETCH preserva el orden entre las llaves, así que $\text{SKETCH}(q)$ podría caer en cualquier lugar, no necesariamente en su sucesor o predecesor.

¿Qué hacemos?

Supongamos que $\text{SKETCH}(x_i) \leq \text{SKETCH}(q) < \text{SKETCH}(x_{i+1})$.
Tomamos el *longest common prefix* entre q y x_i o x_{i+1} .

Consultas



¿Cómo buscamos el sucesor o predecesor de un valor q ?

Podemos pensar en calcular $\text{SKETCH}(q)$ y luego buscarlo en la estructura. Sorpresa: SKETCH preserva el orden entre las llaves, así que $\text{SKETCH}(q)$ podría caer en cualquier lugar, no necesariamente en su sucesor o predecesor.

¿Qué hacemos?

Supongamos que $\text{SKETCH}(x_i) \leq \text{SKETCH}(q) < \text{SKETCH}(x_{i+1})$.

Tomamos el *longest common prefix* entre q y x_i o x_{i+1} .

El nodo al que lleguemos es en donde la búsqueda *falla*, así que podremos decidir qué hacer a continuación dependiendo del escenario.

Consultas



¿Cómo buscamos el sucesor o predecesor de un valor q ?

Podemos pensar en calcular $\text{SKETCH}(q)$ y luego buscarlo en la estructura. Sorpresa: SKETCH preserva el orden entre las llaves, así que $\text{SKETCH}(q)$ podría caer en cualquier lugar, no necesariamente en su sucesor o predecesor.

¿Qué hacemos?

Supongamos que $\text{SKETCH}(x_i) \leq \text{SKETCH}(q) < \text{SKETCH}(x_{i+1})$.

Tomamos el *longest common prefix* entre q y x_i o x_{i+1} .

El nodo al que lleguemos es en donde la búsqueda *falla*, así que podremos decidir qué hacer a continuación dependiendo del escenario.

Un ejemplo interesante es con las llaves 0, 2, 12, 15 y $q = 5$.

Resumen de la sesión



Resumen de la sesión



- ▶ Aprendimos un poco del modelo RAM AC^0 .

Resumen de la sesión



- ▶ Aprendimos un poco del modelo RAM AC^0 .
- ▶ Aprendimos sobre Fusion Trees.

Gracias